

CMSC 478: Homework 2

Christian de Guzman

April 8, 2026

Comparative Analysis

Q1: The majority-class trap. What is the test accuracy of both models? What is the recall on the fraud class? Explain why high accuracy coexists with near-zero fraud recall. Compute and report the metrics of the trivial “always predict 0” baseline.

The test accuracy for perceptron scratch is 0.998990541815708 and for perceptron sklearn is 0.9993328798086418. Both scratch and sklearn have high accuracy. The recall for perceptron scratch is 0.8367346938775511 and perceptron sklearn 0.8061224489795918.

Q2: Convergence analysis. Plot the number of misclassifications per epoch from Part A. Does the error converge to zero? Characterize the curve: is it monotonically decreasing, oscillating, or flat? Explain what this reveals about the geometry of the dataset in 29 dimensions.

The number of misclassifications does not converge to zero. It oscillates across epochs. This reveals that the data set is not linearly separable in 29 dimensional space.

Q3: Precision–Recall tradeoff. Using your from-scratch model’s raw linear output ($w \cdot x + b$) as a continuous score, plot the Precision–Recall curve. Report the AUPRC. Compare it to a random baseline (AUPRC $x \approx 0.00172$).

The scratch model’s AUPRC is 0.6985884882203427 compared to the random baseline (AUPRC $x \approx 0.00172$). It shows that the perceptron’s linear scores provides meaningful ranking of fraud vs non fraud despite the imbalance.

Q4: Effect of class-weight rebalancing. Compare the fraud-class F1-score from Part B (default weights) to the bonus run with class weight=0:1, 1:578. Explain the precision/ recall tradeoff that class-weight adjustments introduce. Does rebalancing help or hurt overall?

The F1-score for perceptron sklearn is 0.7707317073170732 and its precision is 0.738317757009345. While the F1-score with class-weight is 0.0448055145248646 and precision is 0.022956609485368314. The recall for the without class-weight adjustments is 0.8061224489795918. The recall then jumps to 0.9285714285714286

with class-weight adjustments. Class-weight increases recall but reduces precision. The model captures most actual fraud cases but also produce large number of false positives.

Q5: Weight analysis. Which PCA features (V1–V28) receive the largest absolute weights? Do both implementations agree on the top-5 features? What does this suggest about which latent transaction dimensions are most discriminative for fraud?

The one with the largest weights represent the most influential PCA component. Agreement between two implementations confirms correctness. The top PCA corresponds to the latent transaction patterns most correlated with fraud.

Q6: Fundamental limitations. The perceptron achieves a hyperplane in R29. State of the art methods on this dataset use ensemble trees or deep networks with focal loss and achieve $x \approx 0.85$ F1 on the fraud class. Explain (a) why a single hyperplane is insufficient, (b) what non-linear extensions would help, and (c) what role resampling techniques (SMOTE, undersampling) could play.

a) Fraud detection is not linearly separable. Fraud patterns are scattered through PCA space in clusters that cannot be captured by a single linear boundary. b) A non-linear extension that would help is Kernel SVM. It implicitly maps features to a higher dimensional space where classes can be separable. c) SMOTE and undersampling can be combined with the non-linear models for maximum effect. SMOTE creates synthetic fraud examples to balance training and undersampling reduces majority class dominance.

Q7: Computational cost. Report wall-clock time per epoch for Part A (from-scratch loop) versus total training time for Part B (sklearn). What optimizations does sklearn use that your loop does not?

For perceptron scratch, the Epoch time is: 0.36s, 0.73s, 1.09s, ..., 18.62s. For perceptron sklearn, the total training time is 1.54s. Sklearn perceptron is faster because it is most likely been optimized. Many of scikit-learn algorithms are written in Cython, compiling Python-like code into C.

https://scikit-learn.org/stable/computing/computational_performance.html